

Graph Theory — Mid-Sem Revision

CS-5136 · Topic-wise summary, pages 1–194 (Day 1 – Day 10) · formula & application focus

Contents

How to Use This Document	3
Chapter 1 — The Shortest Superstring Problem	3
1.1 Motivation	3
1.2 Definitions	3
1.3 The reduction: SSP as a shortest Hamiltonian path	4
1.4 Worked example	4
1.5 Which direction does the reduction actually prove?	4
1.6 NP-hardness of SSP (reduction from Vertex Cover — full proof, not skip-listed)	5
1.7 Common exam traps	5
Chapter 2 — The Graph Abstraction	6
2.1 The formal definition	6
2.2 Basic terminology	6
2.3 Parallel edges, self-loops, and simple graphs	6
2.4 Finite, infinite, null graphs; directed graphs	6
2.5 More vertex types, order, size, neighbourhood	7
2.6 The Handshaking Lemma	7
2.7 Maximum edges in a simple graph (full proof — not skip-listed)	7
2.8 A gallery of special graphs	7
2.9 A homework theorem — infinite edges force infinite parallel edges	8
2.10 Worked examples & common exam traps	8
Chapter 3 — Graph Operations	9
3.1 The four operations	9
3.2 Counting formulas (the numerical workhorse)	9
3.3 Worked examples	9
3.4 Common exam traps	10
Chapter 4 — Isomorphism, Automorphism, Orbit	11
4.1 Isomorphism	11
4.2 How to actually settle isomorphism on a numerical question	11
4.3 Automorphism and orbit	12
4.4 Aside — vertex-transitive graphs	12
4.5 Common exam traps	12
Chapter 5 — Subgraphs & Five More Operations	13
5.1 Subgraph types	13
5.2 Five more operations	13
5.3 Bonus formula — number of ways to decompose m edges into two pieces	14
5.4 Common exam traps	14

Chapter 6 — Degrees: the Basics and Two Theorems	15
6.1 Definitions and the sandwich inequality	15
6.2 Theorem — two vertices always share a degree	15
6.3 Average degree forces a denser piece (not in the skip-list — full idea below)	15
6.4 Common exam traps	15
Chapter 7 — Degree Sequences & the Havel-Hakimi Test	16
7.1 Definitions	16
7.2 Two instant necessary checks (do these BEFORE running the algorithm)	16
7.3 The Havel-Hakimi Test	16
7.4 Worked examples (fully solved, QB Day 5 style)	16
7.5 Common exam traps	17
Chapter 8 — Walks, Paths, Distance & Connectivity	18
8.1 The walk / trail / path hierarchy	18
8.2 Length, distance, diameter	18
8.3 Any walk can be shortened to a path (Prop 3.1 — not proof-skipped, short idea)	18
8.4 Connectedness is an equivalence relation	18
8.5 Two connectedness theorems	18
8.6 Other useful facts from this chapter (full proofs — short, worth knowing)	19
8.7 Common exam traps	19
Chapter 9 — Regular Graphs, Girth, Components & Instant Insanity	20
9.1 When does an r -regular graph on n vertices exist?	20
9.2 Embedding any graph inside a regular one (not skip-listed)	20
9.3 Cycles, girth, components	20
9.4 Max edges with k components	20
9.5 Exactly two odd-degree vertices $\Rightarrow$ a path joins them (not skip-listed)	21
9.6 Instant Insanity (the Four-Cubes Puzzle)	21
9.7 Bonus fact: $G - v$ is 3-regular $\Rightarrow V(G) $ is odd	21
9.8 Common exam traps	21
Chapter 10 — Euler Graphs, Unicursal Graphs & Fleury’s Algorithm	22
10.1 Euler circuits and Euler’s theorem	22
10.2 A closed trail always decomposes into cycles (not skip-listed)	22
10.3 Fleury’s Algorithm — how to actually find the tour	22
10.4 Unicursal graphs (Euler trails)	22
10.5 Generalisation: $2k$ odd vertices $\Rightarrow k$ unicursal pieces (not skip-listed)	23
10.6 Arbitrarily traceable graphs	23
10.7 Join of two graphs — Eulerian condition (not skip-listed)	23
10.8 Common exam traps	23
Chapter 11 — Hamiltonian Graphs	24
11.1 Definitions and quick one-directional facts	24
11.2 Necessary condition — the component-counting test (mainly used to DISPROVE Hamiltonicity)	24
11.3 Ore’s Theorem	24
11.4 Closure Theorem	24
11.5 Degree-Counting Theorem (a second sufficient condition)	25
11.6 The Petersen graph — a graph where “nice” invariants can still fail Hamiltonicity	25
11.7 Counting Hamiltonian cycles that decompose K_n (odd n) — not skip-listed, full construction	25
11.8 Common exam traps	25
Chapter 12 — Master Formula Sheet (last-minute lookup)	27

How to Use This Document

This is a **self-contained** revision guide. You should not need to go back to the three source PDFs after reading this — except for one thing: **full proofs**. Wherever a theorem’s proof already appears in your separate **17-Proof “Main Proofs” PDF**, this guide states the theorem and shows you exactly *how to use it on a numerical question*, but does not re-derive the proof — read the proof there if you need it. Every such theorem is marked like this:

[Proof lives in your Main Proofs PDF] — item #N. Here you only need the statement and how to apply it.

Everything else — definitions, formulas, worked numericals, algorithms — is derived here in full, because these are the tools an exam actually asks you to *use*, not recite.

Scope. This covers pages 1–194 of your slides in full: everything from Day 1 (the Shortest Superstring Problem) through Day 10 (Hamiltonian Graphs, Part 2) — your **entire** mid-sem syllabus, nothing skipped. It is organised **topic-wise**, not day-wise, because several days share one theme (e.g. Hamiltonian graphs spans Day 9–10) and a few topics from one day are more naturally grouped with another.

How it’s organised. Eleven chapters, each self-contained: definition → formula box → worked numerical example(s) → common exam traps. Chapter 12 is a one-page master formula sheet for last-minute lookup.

Note on scope. This guide sticks strictly to material that is actually in your course slides (the 230-slide CS-5136 deck, pages 1–194). Matching theory and the “stable set / vertex cover” bridging vocabulary from an earlier pass of this guide have been **removed** — they came from an external reference book (Held, *Combinatorial Optimization*), not from your slides, and are not part of this course’s syllabus.

Chapter 1 — The Shortest Superstring Problem

Why you need this. This is the motivating story for the whole course: a real computational problem (genome assembly) gets turned into a graph problem (shortest Hamiltonian path), which is exactly the move you’ll make again and again from here on. It’s also where the course’s very first NP-hardness result comes from.

1.1 Motivation

Real problems that motivate modelling with graphs: railway/train scheduling (timetables mod 60, stock circulation as a flow problem, a circular route needing every segment traversed exactly once — an **Eulerian circuit**, Chapter 11), and DNA fragment assembly. Sequencing machines only read short fragments (“reads”) of a genome, never the whole thing at once — so the genome is sheared into millions of overlapping reads, and the computational challenge is to reassemble them into one full sequence.

1.2 Definitions

Shortest Superstring Problem (SSP). Input: strings $s_1, \dots, s_n$. Output: a shortest string s that contains every s_i as a substring (not necessarily at the start).

Overlap function. $\text{overlap}(s_i, s_j) :=$ the length of the longest prefix of s_j that matches a suffix of s_i .

Beginner trap. Overlap is **not symmetric**: $\text{overlap}(s_i, s_j) \neq \text{overlap}(s_j, s_i)$ in general — always ask “suffix of **which** string matches prefix of **which** string.” This asymmetry is exactly why the reduction graph below must be **directed**.

Gluing s_i and s_j along their overlap saves $\text{overlap}(s_i, s_j)$ characters: the combined string has length $|s_i| + |s_j| - \text{overlap}(s_i, s_j)$ instead of $|s_i| + |s_j|$.

Why not just concatenate everything? It always works, but is almost never shortest. E.g. all 8 length-3 binary strings concatenated give length $8 \times 3 = 24$, but the single string 0001110100 (length 10) already contains all 8 as substrings by reusing overlaps.

1.3 The reduction: SSP as a shortest Hamiltonian path

Construction. Build a **directed**, complete graph G : one vertex per string s_i ; for every ordered pair (i, j) with $i \neq j$, an arc $s_i \rightarrow s_j$ weighted

$$d(s_i, s_j) := |s_j| - \text{overlap}(s_i, s_j) \quad (\text{always} \geq 0)$$

— the number of *brand-new* characters s_j contributes once glued onto s_i . (Weighting by raw overlap instead would force a *maximum-weight* Hamiltonian path, an unusual framing; weighting by d gives the standard *shortest* Hamiltonian path framing.)

Theorem (not proof-skipped — short and worth having in full). Shortest superstring = shortest Hamiltonian path w.r.t. weights d , in this complete directed graph.

Proof idea: fix an ordering (a Hamiltonian path) $s_{i_1} \rightarrow s_{i_2} \rightarrow \dots \rightarrow s_{i_n}$. Building the superstring left to right, each next string contributes exactly $d(s_{i_k}, s_{i_{k+1}})$ new trailing characters (by definition of d), so the total length telescopes to $|s_{i_1}| + \sum d(s_{i_k}, s_{i_{k+1}})$ — i.e. (first string’s length) + (total arc weight of the path). Minimising over all orderings and starting vertices is exactly minimising total arc weight over all Hamiltonian paths — a **shortest Hamiltonian path** problem (this specific flavour — visit every vertex once, minimise weight, no return to start — is called the **Path-TSP**). ■

1.4 Worked example

$S = \{\text{ATC}, \text{CCA}, \text{CAG}, \text{TCC}, \text{AGT}\}$. Pairwise overlaps (row→col):

overlap	ATC	CCA	CAG	TCC	AGT
ATC	—	1	1	2	0
CCA	1	—	2	0	1
CAG	0	0	—	0	2
TCC	0	2	1	—	0
AGT	0	0	0	1	—

The order $\text{ATC} \rightarrow \text{TCC} \rightarrow \text{CCA} \rightarrow \text{CAG} \rightarrow \text{AGT}$ uses overlap 2 at every step (the maximum possible), giving superstring **ATCCAGT** (length 7; total lengths 15, total overlap 8, $15 - 8 = 7$). Compare the alphabetical order $\text{AGT} \rightarrow \text{ATC} \rightarrow \text{CAG} \rightarrow \text{CCA} \rightarrow \text{TCC}$: gives **AGTATCAGCCATCC**, length 14 — **twice as long**. Ordering is everything.

1.5 Which direction does the reduction actually prove?

What Section 1.3 builds is: SSP can be **solved by** solving a shortest-Hamiltonian-path instance, written $\text{SSP} \leq_p \text{Path-TSP}$ (“SSP is no harder than Path-TSP”). This **alone does not** show SSP is NP-hard — that needs the reduction in the **other** direction (an arbitrary hard instance encoded as an SSP instance). That’s Section 1.6.

1.6 NP-hardness of SSP (reduction from Vertex Cover — full proof, not skip-listed)

Theorem. The Shortest Superstring Problem is NP-hard.

Construction. Reduce from Vertex Cover: given a graph $H = (V, E)$, $|V| = n$, $|E| = m$, and target k , decide if H has a vertex cover of size $\leq k$. Use one alphabet letter per vertex of H . For every edge $\{a, b\} \in E$, put two length-4 strings into the SSP instance: $abab$ and $baba$. So $|S| = 2m$.

Claim. H has a vertex cover of size $\leq k \iff S$ has a superstring of length $\leq 4m + k$.

($\Rightarrow$). Let $C = \{c_1, \dots, c_k\}$ be a vertex cover; assign every edge to one of its covering endpoints. For an edge $\{a, b\}$ assigned to covering vertex a : $\text{overlap}(abab, baba) = 3$ (last 3 letters of $abab$ = first 3 of $baba$ = “bab”), merging to $ababa$ — a length-5 token starting **and** ending with the covering letter. This is the key trick: it lets you **chain** every edge assigned to the same covering vertex, overlapping by exactly 1 letter between consecutive tokens. If $c \in C$ covers k_c edges, its chain has length $5 + 4(k_c - 1) = 4k_c + 1$. Concatenating the (non-overlapping) chains for all of C ’s vertices gives total length $\sum_{c \in C} (4k_c + 1) = 4m + k$ (since every one of the m edges is assigned to exactly one vertex of C).

($\Leftarrow$, **sketch**). Any superstring of S can be rearranged, without increasing length, into exactly this “chains of length-5 tokens” shape (the 4-letter strings can only overlap substantially with strings sharing a letter). A superstring of length $\leq 4m + k$ then forces some set of $\leq k$ letters to appear as “chain centres” — exactly a vertex cover of size $\leq k$.

Worked example. H = path 1–2–3–4, $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$, $m = 3$. Minimum vertex cover $C = \{2, 3\}$, $k = 2$. Assign $\{1, 2\} \rightarrow 2$, $\{2, 3\} \rightarrow 2$, $\{3, 4\} \rightarrow 3$.

Covering vertex	Assigned edges	Chained token
2	$\{1, 2\}, \{2, 3\}$	$21212 \oplus 23232 = 212123232$ (len 9)
3	$\{3, 4\}$	34343 (len 5)

Superstring: $212123232 \cdot 34343 = 21212323234343$, length $14 = 4(3) + 2$ — matching $4m + k$ exactly, confirming the construction encodes the vertex-cover number precisely. ■

This closes the loop: $\text{SSP} \leq_p \text{Path-TSP}$ (Section 1.3) and $\text{Vertex-Cover} \leq_p \text{SSP}$ (above) together show SSP sits among the NP-hard problems.

1.7 Common exam traps

- Overlap is **directional** — always double-check which string’s suffix vs. which string’s prefix.
- The weight for the reduction graph is $d(s_i, s_j) = |s_j| - \text{overlap}(s_i, s_j)$, **not** the raw overlap itself.
- $\text{SSP} \leq_p \text{Path-TSP}$ shows SSP is *easy if* Path-TSP is easy — it does **not**, by itself, prove SSP is NP-hard. NP-hardness needs the separate Vertex-Cover reduction (Section 1.6).

Chapter 2 — The Graph Abstraction

Why you need this. Every later chapter’s vocabulary comes from here. This chapter also has the course’s first proper theorems (Handshaking Lemma, max-edges bound) and the “gallery” of named graphs (K_n , C_n , P_n , W_n , ...) you’ll keep referencing.

2.1 The formal definition

Definition. A graph $G = (V, E)$ consists of a set $V = \{v_1, v_2, \dots\}$ of vertices and a set $E = \{e_1, e_2, \dots\}$ of edges, where every edge is written $e_i = (v_p, v_q)$ and connects v_p, v_q . In plain words: dots (vertices) and lines between some pairs of them (edges).

Running example G_1 (reused throughout this chapter): $V = \{v_1, \dots, v_5\}$, $E = \{e_1, \dots, e_7\}$, with $e_2 = (v_2, v_4)$, and e_1 a self-loop at v_2 ; e_4, e_5 both join v_1, v_3 (a parallel pair).

2.2 Basic terminology

Term	Meaning
Adjacent vertices	Some edge connects them.
Edge incident on a vertex	That vertex is one of its endpoints.
Degree $d(v)$	Number of edges incident on v (a self-loop counts twice).

Worked example (G_1). v_1, v_3 are adjacent (twice over, via e_4, e_5). Edges e_4, e_5 are both incident on v_1 . Degree of v_2 is **4**: one each for e_2, e_3 , plus **two** for the self-loop e_1 .

2.3 Parallel edges, self-loops, and simple graphs

- **Parallel edges:** two (or more) edges connecting the same pair of vertices.
- **Self-loop:** an edge from a vertex back to itself.
- **Simple graph:** no parallel edges, no self-loops — **the default assumption from here on**, unless a problem says otherwise.

In G_1 : e_4, e_5 are parallel edges (both join v_1, v_3), e_1 is a self-loop at v_2 . $G_2 = “G_1$ with the loop and one parallel edge removed” is simple: 5 vertices, 5 edges, no parallel edges, no loops.

Beginner note. An edge can usually be written as an unordered vertex pair (e.g. $e_2 = (v_2, v_4)$) — except that parallel edges break this: e_4, e_5 are different edges but the *same pair* (v_1, v_3) , which is exactly why edges need their own names once parallel edges are allowed.

2.4 Finite, infinite, null graphs; directed graphs

- **Infinite graph:** infinitely many vertices. **Finite:** otherwise. **Null (trivial) graph:** nonempty vertex set, empty edge set.
- **Directed graph (digraph):** every edge has a direction. **In-degree / out-degree:** edges pointing in / out of a vertex.

Worked example (digraph). For a 5-vertex digraph with edges $e_1, \dots, e_5$:

	v_1	v_2	v_3	v_4	v_5
in-degree	1	1	0	2	1
out-degree	1	1	2	1	0

v_3 has in-degree 0 — a natural “**source**.” v_5 has out-degree 0 — a natural “**sink**.”

2.5 More vertex types, order, size, neighbourhood

- **Isolated vertex:** no incident edges. **Pendant vertex:** degree exactly 1.
- **Order** $|G|$ = number of vertices. **Size** $\|G\|$ = number of edges.
- **Neighbourhood** $N(v)$: the set of v ’s neighbours (a *set*, so a parallel-edge neighbour is listed once). $d(v)$ counts *edges* (with multiplicity); $N(v)$ counts distinct *neighbours*.

Worked example (G_1). $|G_1| = 5$, $\|G_1\| = 7$, $N(v_1) = \{v_2, v_3\}$ (note: v_3 appears once in $N(v_1)$ even though two parallel edges connect them), $d(v_1) = 3$ (e_3, e_4, e_5).

2.6 The Handshaking Lemma

[Proof lives in your Main Proofs PDF] — item #1.

$$\sum_v d(v) = 2\|G\|$$

One-line idea: every ordinary edge contributes exactly 1 to each of its two endpoints’ degree (total 2); every self-loop contributes 2 to its one vertex’s degree (also total 2) — so every edge contributes exactly 2 to the grand total, summed over all $\|G\|$ edges.

[Proof lives in your Main Proofs PDF] — item #2 (Corollary). **Every graph has an even number of odd-degree vertices.** (Split into V_{odd} , V_{even} ; the even part’s sum is even, so by Handshaking the odd part’s sum is also even — and a sum of odd numbers is even only when there’s an even count of them.)

Worked example (G_1). $d(v_1)=3, d(v_2)=4, d(v_3)=3, d(v_4)=3, d(v_5)=1$. Sum = 14 = $2(7) = 2\|G_1\|$. Odd-degree vertices: v_1, v_3, v_4, v_5 — that’s 4, an even count. Both checks pass.

2.7 Maximum edges in a simple graph (full proof — not skip-listed)

$$\text{a simple graph on } n \text{ vertices has at most } \binom{n}{2} = \frac{n(n-1)}{2} \text{ edges}$$

Proof by induction on n . Base case $n = 1$: a single vertex has no possible edge, and $\binom{1}{2} = 0$. OK Inductive step: let G have $k + 1$ vertices. Delete any vertex v (with its edges) to get a simple G' on k vertices; by hypothesis $\|G'\| \leq \binom{k}{2}$. Since G is simple, v has no self-loop and no parallel edges, so $d_G(v) \leq k$. Then $\|G\| = \|G'\| + d_G(v) \leq \binom{k}{2} + k = \frac{k(k+1)}{2} = \binom{k+1}{2}$, exactly the formula for $n = k + 1$. ■

Slicker way: every edge of a simple graph is a distinct unordered pair of vertices, and there are exactly $\binom{n}{2}$ such pairs available — so $\|G\| \leq \binom{n}{2}$ immediately, with equality exactly when $G = K_n$ (the complete graph, Section 2.8).

2.8 A gallery of special graphs

Graph	Definition
K_n (complete)	Every vertex adjacent to every other. $\ K_n\ = \binom{n}{2}$ exactly — the graph that <i>achieves</i> Section 2.7’s bound.

Graph	Definition
C_n (cycle)	Every vertex has degree exactly 2.
P_n (path)	A cycle graph with one edge removed.
W_n (wheel)	A cycle plus one new hub vertex joined to every cycle vertex.
r -regular	Every vertex has the same degree r .
Bipartite	V splits into two disjoint parts so every edge has one end in each part.

Worked example (regularity checks). K_4 : every vertex touches the other 3 $\Rightarrow$ **3-regular**. C_6 : every vertex touches exactly 2 neighbours $\Rightarrow$ **2-regular**. (The Petersen graph, revisited in Chapter 11.6, is 3-regular with 10 vertices, 15 edges.)

2.9 A homework theorem — infinite edges force infinite parallel edges

[Proof lives in your Main Proofs PDF] — item #16. A graph with finitely many vertices but infinitely many edges must have at least one pair of vertices joined by infinitely many parallel edges. (*Pure pigeonhole: finitely many vertex-pairs, infinitely many edges to distribute among them — some pair gets infinitely many.*)

Wording subtlety. This theorem’s “infinite graph” means *infinitely many edges* (a multigraph with finitely many vertices but unboundedly many parallel edges) — a different sense from Section 2.4’s “infinite graph” (infinitely many *vertices*). Keep both readings in mind; which one applies depends on context.

2.10 Worked examples & common exam traps

Trap 1. “No isolated vertices” is sometimes mistaken for “connected” — they’re different (see Chapter 8.1 for the correct definition and counterexample).

Trap 2. Don’t forget self-loops count **twice** toward degree — a very common off-by-one in Handshaking-Lemma computations.

Chapter 3 — Graph Operations

Why you need this. Given two graphs $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$, four standard operations combine them into a new graph. Every exam question here is really just **careful bookkeeping** — get the definitions exactly right and the arithmetic is trivial.

3.1 The four operations

Operation	Vertex set	Edge set	Key idea
Union $G_1 \cup G_2$	$V_1 \cup V_2$	$E_1 \cup E_2$	Edges with the <i>same name</i> in both graphs appear once; edges with <i>different names</i> but the same endpoints survive as parallel edges .
Intersection $G_1 \cap G_2$	$V_1 \cap V_2$	edges that are literally the <i>same edge</i> (same name/identity) in both	Same endpoints in both graphs is not enough — it must be the same edge.
Ring sum $G_1 \oplus G_2$	$V_1 \cup V_2$	$(E_1 \setminus E_2) \cup (E_2 \setminus E_1)$	Symmetric difference — edges in <i>exactly one</i> of the two graphs.
Cartesian product $G_1 \boxtimes G_2$	$V_1 \times V_2$	$(u, v) \sim (x, y)$ iff [$u = x$ and $vy \in E_2$] or [$v = y$ and $ux \in E_1$]	“Move along one coordinate at a time , the way the corresponding graph allows.”

Beginner tip. Union/intersection/ring-sum are commutative and come straight from set theory (they’re just set union/intersection/symmetric-difference applied to edge sets). The Cartesian product is commutative only *up to isomorphism* (swap the coordinates).

3.2 Counting formulas (the numerical workhorse)

Let $n_i = |V_i|$, $m_i = \|G_i\|$.

$$\begin{aligned}
 |V(G_1 \cup G_2)| &= n_1 + n_2 - |V_1 \cap V_2| & \|G_1 \cup G_2\| &= m_1 + m_2 - \|G_1 \cap G_2\| \\
 \|G_1 \oplus G_2\| &= m_1 + m_2 - 2\|G_1 \cap G_2\| \\
 |V(G_1 \boxtimes G_2)| &= n_1 n_2 & \|G_1 \boxtimes G_2\| &= n_1 m_2 + n_2 m_1
 \end{aligned}$$

Why the ring-sum formula has a “2”: start from $E_1 \cup E_2$ (which already double-subtracts the shared part once), then **remove** the shared edges completely (not just de-duplicate) — that’s a second subtraction.

3.3 Worked examples

Example 1 (union/intersection/ring-sum). $n_1 = n_2 = 5$, $m_1 = m_2 = 6$, $|V_1 \cap V_2| = 4$, $\|G_1 \cap G_2\| = 2$.

$$|V_3| = 5 + 5 - 4 = 6 \quad \|G_1 \cup G_2\| = 6 + 6 - 2 = 10 \quad \|G_1 \oplus G_2\| = 6 + 6 - 2(2) = 8$$

Example 2 (Cartesian product). $G_1 = P_3$ (path $u_1-v_1-w_1$: $n_1 = 3, m_1 = 2$), $G_2 = P_2$ (single edge: $n_2 = 2, m_2 = 1$).

$$|V| = 3 \times 2 = 6 \quad \|G_1 \boxtimes G_2\| = 3(1) + 2(2) = 3 + 4 = 7$$

This is the 3×2 “ladder” grid: 3 horizontal edges (from G_2 ’s single edge, copied at each of the 3 u -levels) + 4 vertical edges (from G_1 ’s 2 edges, copied at each of the 2 v -levels).

Example 3 ($P_2 \boxtimes P_2$). $n_1 = n_2 = 2, m_1 = m_2 = 1$: vertices = 4, edges = $2(1) + 2(1) = 4$ — every vertex degree 2 $\Rightarrow$ this is C_4 (a square).

3.4 Common exam traps

- **Degree in a union is not just additive.** $d_{G_1 \cup G_2}(v) = d_{G_1}(v) + d_{G_2}(v) - s(v)$, where $s(v)$ is the number of edges at v shared by both graphs.
- Intersection can leave a vertex **isolated** — it still belongs to $V_1 \cap V_2$ even if none of its edges survive.
- Decomposition (Chapter 6) is the *reverse* of union: splitting one graph’s edges into two edge-disjoint pieces.

Chapter 4 — Isomorphism, Automorphism, Orbit

Why you need this. “Are these two graphs really different, or just drawn/labelled differently?” This is one of the most common numerical/qualitative question types.

4.1 Isomorphism

Definition. $G \cong H$ if there’s a bijection $\varphi : V(G) \rightarrow V(H)$ such that $uv \in E(G) \iff \varphi(u)\varphi(v) \in E(H)$.

Necessary (never sufficient) conditions — check these first, fastest to rule things out:

1. Same $|V|$
2. Same $|E|$
3. Same degree sequence

If **any** of these differ $\Rightarrow$ **not isomorphic**, instantly. If all three match, you must dig further (see 4.2).

A correction worth knowing. “Testing graph isomorphism is NP-complete” is a common myth — it’s **not known** to be NP-complete (Babai’s 2015 algorithm runs in quasipolynomial time, far below what NP-complete problems are believed to allow). State it as: *no polynomial algorithm is known, and it’s not proven/believed to be NP-complete either.*

4.2 How to actually settle isomorphism on a numerical question

To disprove ($G \not\cong H$): find **any one invariant** that differs. In rough order of how often it’s the fastest disqualifier:

- Vertex/edge counts, degree sequence (Section 4.1’s checklist)
- **Neighbour-degree pattern** at same-degree vertices (two trees can share every count above yet differ here — Example A below)
- **Bipartiteness** (one has an odd cycle, the other doesn’t — an isomorphism-invariant property)
- **Connectedness** (one is connected, the other splits into components)

To prove ($G \cong H$): exhibit an explicit bijection φ and verify every edge maps correctly (or, if the two pictures are “obviously the same graph drawn twice,” the identity map on matching labels already works).

Example A — same numbers, still not isomorphic. Two trees T_1, T_2 : both 6 vertices, 5 edges, degree sequence 1, 1, 1, 1, 2, 3. But at the degree-3 vertex: in T_1 its 3 neighbours have degrees $\{2, 1, 1\}$; in T_2 , $\{2, 2, 1\}$. No relabelling preserves this, so $T_1 \not\cong T_2$.

Example B (QB Day4 Q1 — the cube vs. the octagon+diagonals). G_1 = cube graph Q_3 (outer 4-cycle + inner 4-cycle + 4 connecting spokes); G_2 = an 8-cycle plus its 4 main diagonals. Both 8 vertices, 12 edges, 3-regular — **all three necessary checks pass**. Tie-breaker: G_1 is **bipartite** (colour “outer vs inner”, every edge type crosses colours). G_2 contains the 5-cycle $A-B-C-D-E-A$ (4 boundary edges + 1 diagonal) — an **odd** cycle, so G_2 is not bipartite. Bipartiteness is preserved under isomorphism $\Rightarrow G_1 \not\cong G_2$.

Example C (QB Day4 Q2 — Deo 2-4). $G_1 = C_6$ (one hexagon, connected), $G_2 = 2C_3$ (two disjoint triangles). Same vertex count (6), same edge count (6), same degree sequence (all 2’s) — but G_1 is connected and G_2 isn’t. Connectedness is an isomorphism invariant $\Rightarrow$ not isomorphic.

Example D — any two connected 2-regular simple graphs on n vertices are isomorphic (both forced to be C_n : start walking from any vertex, at each degree-2 vertex there’s exactly one “new” edge to continue on, so the walk traces a unique cycle through all n vertices).

4.3 Automorphism and orbit

- **Automorphism:** an isomorphism from G to **itself**. The identity map always works. All automorphisms of G together form a group $\text{Aut}(G)$ (under composition).
- **Orbit of v :** the set of vertices v can be mapped to by *some* automorphism. Two vertices in the same orbit are “structurally interchangeable” (called *similar*).

Worked example. H : a triangle $\{v_1, v_2, v_3\}$ bridged at v_3 - v_4 to a “cherry” $\{v_4, v_5, v_6\}$ (triangle + pendant pair). Automorphisms: $\varphi_1 = \epsilon$ (identity), $\varphi_2 = (v_1 v_2)$, $\varphi_3 = (v_5 v_6)$, $\varphi_4 = (v_1 v_2)(v_5 v_6)$ — 4 total, forming the **Klein four-group** (every element is its own inverse). Orbits: $\{v_1, v_2\}$, $\{v_3\}$, $\{v_4\}$, $\{v_5, v_6\}$ — 4 orbits. (*Why v_1, v_2 can swap: they play identical roles in the triangle. Why v_3, v_4 can't move: each is the unique vertex of its particular degree-3 role.*)

4.4 Aside — vertex-transitive graphs

[Proof lives in your Main Proofs PDF] — item #17. Statement only: **every vertex-transitive graph is regular, but the converse is false**. (Vertex-transitive means: for any two vertices u, v , some automorphism sends $u \mapsto v$ — a much stronger symmetry requirement than just “some vertices share an orbit.”) Counterexample to the converse: $K_3 \sqcup C_4$ (disjoint triangle + square) — every vertex has degree 2 (regular!), but no automorphism can ever map a triangle-vertex to a square-vertex (automorphisms preserve which component you're in, and the components have different sizes).

4.5 Common exam traps

- Matching necessary conditions is **not a proof** of isomorphism — always keep digging or exhibit the bijection.
- An automorphism must preserve degree — a map sending a degree-2 vertex to a degree-3 vertex is automatically disqualified, no further check needed.
- “ G and H have the same degree sequence” is the *weakest* useful check — don't stop there.

Chapter 5 — Subgraphs & Five More Operations

5.1 Subgraph types

Type	Rule	Notes
Subgraph $H \subseteq G$	$V' \subseteq V, E' \subseteq E$	Any piece, kept whole.
Proper	$V' \subsetneq V$ or $E' \subsetneq E$	At least one strict subset.
Spanning	$V' = V$	All original vertices kept; only edges may shrink.
Vertex-induced	pick V' ; auto-include every G -edge with both ends in V'	You cannot “leave an edge out” if both its endpoints were chosen.
Edge-induced	pick E' ; $V' =$ exactly the endpoints appearing in E'	Vertex set is <i>forced</i> by the edge choice.

Worked example. G : vertices $\{u, v, w, x, y\}$, edges $uv, vw, vx(= e), wy, xy(= e')$ — a 4-cycle v, w, y, x with pendant u on v . - Vertex-induced on $\{v, w, x\}$: must include vw and vx (both endpoints chosen); wx isn't a G -edge at all so it's simply absent. Result: $F = (\{v, w, x\}, \{vw, vx\})$. - Edge-induced by $\{e, e'\} = \{vx, xy\}$: forced vertex set $\{v, x, y\}$ — a path $v-x-y$. - **Exam trap:** the subgraph on $\{u, v, w\}$ with only edge $\{uv\}$ (deliberately dropping vw) is a valid *subgraph* but **not** an induced subgraph, since $vw \in E(G)$ with both ends inside the chosen set.

5.2 Five more operations

Operation	Rule	Worked check
Complement $\overline{G}$	same V ; $uv \in E(\overline{G}) \iff uv \notin E(G)$	$\overline{K_n} =$ null graph (no edges). $\overline{\overline{G}} = G$ always. $C_5 \cong \overline{C_5}$ (C_5 is <i>self-complementary</i> — its complement is a 5-pointed star, which is C_5 traversed in a different order).
Decomposition	$g_1 \cup g_2 = G$ and $g_1 \cap g_2$ has no edges (edges partition; vertices may overlap)	$g_1 = (\{u, v, w, x\}, \{uv, vw, vx\})$, $g_2 = (\{w, x, y\}, \{wy, xy\})$ decomposes the G above.
Deletion	$G - v_i$: remove v_i and every incident edge . $G - e_j$: remove just that edge (vertices stay, even if isolated).	Identity: $G - e_j = G \oplus \{e_j\}$ (ring-sum with the single-edge graph).
Fusion	merge two vertices a, b into one; every edge that touched a or b now touches the merged vertex — can create self-loops (from the ab edge, if it existed) and parallel edges	Degree is <i>conserved</i> : $d(\text{merged}) = d(a) + d(b)$ (the loop counts twice, exactly replacing the lost ab edge's double-count).

Operation	Rule	Worked check
Closure $C(G)$	repeatedly join nonadjacent pairs with $d(u) + d(v) \geq n$, until none remain	Defined here; the actual use of closure is entirely in Chapter 12 (Hamiltonicity) — park this definition for now.

5.3 Bonus formula — number of ways to decompose m edges into two pieces

genuine decompositions of an m -edge graph $= 2^{m-1} - 1$

(Each edge independently goes to g_1 or g_2 : 2^m ordered choices; a decomposition is unordered so divide by 2 $\rightarrow 2^{m-1}$; subtract the 1 degenerate case where all edges land in one part.) Check: $m = 1 \Rightarrow 0$ (can't split one edge); $m = 2 \Rightarrow 1$ (the only split is $\{e_1\}, \{e_2\}$).

5.4 Common exam traps

- Induced subgraph = **no choice** about which edges to keep once the vertex set is fixed. Ordinary subgraph = you may drop edges freely.
- Fusion's self-loop/parallel-edge bookkeeping is the #1 place marks are lost — always redo the degree count as a sanity check (Section 5.2's fusion row).
- " $G - e_j = G \oplus \{e_j\}$ " is a genuinely useful shortcut: it lets you reuse Chapter 3's ring-sum counting formula for simple deletions too.

Chapter 6 — Degrees: the Basics and Two Theorems

6.1 Definitions and the sandwich inequality

- $\delta(G)$ = minimum degree, $\Delta(G)$ = maximum degree.

$$0 \leq \delta(G) \leq \deg(v) \leq \Delta(G) \leq n - 1 \quad \text{for every vertex } v$$

($\Delta(G) \leq n - 1$ because in a simple graph you can have at most one edge to each of the other $n - 1$ vertices.)

A correct definition worth double-checking. “No isolated vertex” is **not** the same as “connected.” Correct definition: G is connected if there’s a path between **every** pair of vertices. Counterexample to the weaker claim: two separate triangles side by side — every vertex has degree 2 (none isolated), yet the graph is in two unreachable pieces.

6.2 Theorem — two vertices always share a degree

[Proof lives in your Main Proofs PDF] — item #3. Statement: **every simple graph with $n \geq 2$ vertices has two distinct vertices of the same degree.** One-line idea: with n vertices there are only n possible degree values $\{0, \dots, n - 1\}$, but degree-0 and degree- $(n - 1)$ vertices can never coexist (the second would have to be adjacent to the first) — so pigeonhole forces a repeat.

How to use it: in $K_{1,4}$ (star: 1 hub + 4 leaves), the hub has degree 4 (unique), but *all four* leaves share degree 1 — more than the theorem even requires.

6.3 Average degree forces a denser piece (not in the skip-list — full idea below)

$$\bar{d}(G) := \frac{1}{n} \sum_v \deg(v) = \frac{2\|G\|}{n}$$

Theorem. If $\bar{d}(G) \geq 2k$, then G has an **induced subgraph** F with $\delta(F) \geq k + 1$.

Why (the algorithm, which is the numerical skill): repeatedly delete a “weak” vertex (degree $\leq k$) from whatever remains. Each deletion can only *raise* the average degree of what’s left (a careful degree-counting argument shows the average never drops), so this process must terminate at a subgraph with no weak vertex left — i.e. $\delta(F) \geq k + 1$.

Worked example. G = triangle $\{a, b, c\}$ + pendant d on a (edges ab, bc, ca, ad): 4 edges, 4 vertices, $\bar{d} = 2(4)/4 = 2 = 2k$ for $k = 1$. Delete the one weak vertex (d , degree $1 \leq k$): left with the triangle, $\delta = 2 \geq k + 1 = 2$. OK

6.4 Common exam traps

- The two-equal-degree theorem needs $n \geq 2$ and a **simple** graph — it can fail for multigraphs.
- Don’t confuse $\bar{d}(G) \geq 2k$ with “every vertex has degree $\geq 2k$ ” — it’s just the *average*.

Chapter 7 — Degree Sequences & the Havel-Hakimi Test

This is one of the highest-yield numerical topics — practically every exam has a “test this sequence / construct the graph” question.

7.1 Definitions

- **Degree sequence:** the list of every vertex’s degree.
- **Graphical:** a sequence is graphical if it’s the degree sequence of *some* simple graph.

7.2 Two instant necessary checks (do these BEFORE running the algorithm)

1. **Parity (Handshaking):** the sum must be **even**. If odd $\Rightarrow$ reject immediately.
2. **Sandwich bound:** the largest entry must be $\leq n - 1$ (where n = length of the sequence). If violated $\Rightarrow$ reject immediately.

Neither check is sufficient alone — e.g. $3, 3, 3, 1$ passes both (sum = 10 even, max = $3 \leq n - 1 = 3$) yet is **not** graphical: the three degree-3 vertices in a 4-vertex simple graph must each connect to *all* other 3 vertices, forcing the “degree-1” vertex up to degree 3 too. This is exactly why you need the real test below.

7.3 The Havel-Hakimi Test

[Proof lives in your Main Proofs PDF] — item #4. You only need the **algorithm** here — that’s the actual exam skill.

Algorithm. Given a non-increasing sequence $d_1 \geq d_2 \geq \dots \geq d_n$:

1. Remove d_1 (the largest entry).
2. Subtract 1 from each of the **next** d_1 **largest** remaining entries.
3. Re-sort the resulting (shorter) sequence in non-increasing order.
4. Repeat from Step 1 on the new sequence.
5. **Stop:** if you reach all-zeros with no negative ever appearing $\Rightarrow$ **graphical**. If a negative entry ever appears $\Rightarrow$ **not graphical**.

Building the actual graph, once confirmed graphical: work the reduction **in reverse**. The last-added “hub” vertex (largest original degree) connects to exactly the d_1 vertices whose entries were reduced at that step. Peel outward from the smallest sub-problem back to the full sequence.

7.4 Worked examples (fully solved, QB Day 5 style)

Example 1. Is $s = 5, 4, 3, 3, 2, 2, 2, 1, 1, 1$ graphical?

$$\{5, 4, 3, 3, 2, 2, 2, 1, 1, 1\} \rightarrow \{3, 2, 2, 2, 1, 1, 1, 1, 1\} \rightarrow \{1, 1, 1, 1, 1, 1, 1, 1\} \rightarrow \dots \rightarrow \{0, \dots, 0\}$$

No negatives ever appear $\Rightarrow$ **graphical**. (The final all-1’s sequence is trivially 4 disjoint edges — a perfect matching — then build outward: a degree-3 hub joins 3 of those degree-1 vertices, and finally a degree-5 hub joins the degree-3 hub plus 4 more.)

Example 2 (QB Day5 Q1). Sequence $(3, 3, 5, 5, 5, 5)$, $n = 6$ so max allowed degree is 5 — passes the sandwich check. But: the four degree-5 vertices must each connect to *every other* vertex (degree 5 = “adjacent to all 5 others”), so **each** degree-3 vertex is forced to receive an edge from **all four** degree-5 vertices — that’s already degree 4, contradicting its target of 3. **No simple graph exists** with this sequence (confirmed by Havel-Hakimi too). A multigraph could realise it using a parallel edge to soak up the excess. Changing it to $(2, 3, 3, 4, 5, 5)$: run Havel-Hakimi — remove 5, subtract from next 5: $5, 4, 3, 3, 2 \rightarrow 4, 3, 2, 2, 1 \rightarrow$ resort $\rightarrow$ continue... it reduces to all zeros with no negative $\Rightarrow$ **graphical**, a simple graph exists.

Example 3 (QB Day5 Q2). Test $(4, 3, 3, 2, 2, 1, 1)$:

$4, 3, 3, 2, 2, 1, 1 \rightarrow$ (remove 4, subtract from next 4) $\rightarrow 2, 2, 1, 1, 1, 1 \rightarrow 1, 1, 1, 1, 1, 0 \rightarrow 0, 1, 1, 1, 0$ (resort 1, 1)

All zero $\Rightarrow$ **graphical**. One realisation on $v_1, \dots, v_7$ (original degrees $4, 3, 3, 2, 2, 1, 1$): $v_1v_2, v_1v_3, v_1v_4, v_1v_5$ (uses up v_1 's degree 4), v_2v_3, v_2v_6 (degree of v_2 : $1 + 1 + 1 = 3$ OK), v_3v_7 (degree of v_3 : $1 + 1 + 1 = 3$ OK), v_4v_5 (gives v_4, v_5 their remaining degree-1 requirement each), and v_6, v_7 already satisfied — **8 edges total**, matching $\sum d(v)/2 = 16/2 = 8$. OK

Example 4 (QB Day5 Q3 — settle it in one line). Is $(7, 6, 5, 4, 3, 3, 2)$ graphical? $n = 7$ vertices, so max possible degree is $n - 1 = 6$ — but the sequence has a 7. Sandwich bound violated $\Rightarrow$ **not graphical**, no need to run the full reduction.

Example 5 (QB Day5 Q4). In $K_{1,6}$ (hub + 6 leaves), find two equal-degree vertices per Chapter 7's theorem: any **two of the six leaves** (all share degree 1) — this is guaranteed by the “two vertices share a degree” theorem (Section 6.2), Day 5.

7.5 Common exam traps

- Always **re-sort** after each Havel-Hakimi step before repeating — forgetting to re-sort is the #1 arithmetic slip.
- A negative entry appearing mid-reduction means **stop immediately and declare not graphical** — don't keep going.
- “Graphical” only guarantees *some* simple graph exists — it does not mean *every* attempted drawing works; you may need the Havel-Hakimi reverse-construction to actually exhibit one.

Chapter 8 — Walks, Paths, Distance & Connectivity

8.1 The walk / trail / path hierarchy

$$\text{path} \subseteq \text{trail} \subseteq \text{walk}$$

Term	Rule
Walk	Alternating vertex-edge sequence; may repeat both vertices and edges freely.
Trail	A walk with no repeated edge .
Path	An <i>open</i> walk with no repeated vertex (automatically no repeated edge either).
Closed / open	Same start & end vertex, or not.

Worked classification (square a, b, c, d + diagonal, on edges ab, bc, ca, ad): - a, b, c, b, a : reuses edges ab and bc — **walk only**. - a, b, c, a, d : 4 distinct edges but revisits vertex a — **trail, not a path**. - a, b, c, d : all distinct vertices & edges — a genuine **path**.

8.2 Length, distance, diameter

- **Length** of a walk = number of edges (repeats counted).
- **Distance** $d_G(u, v)$ = length of a *shortest* u - v path (that shortest path = a **geodesic**).
- **Diameter** $\text{diam}(G)$ = largest distance between any two vertices of a connected graph.

8.3 Any walk can be shortened to a path (Prop 3.1 — not proof-skipped, short idea)

If a walk a from u to v repeats a vertex ($a_i = a_j$, $i < j$), cut out the loop between them: $(a_0, \dots, a_i, a_{j+1}, \dots, a_k)$ is a **strictly shorter** walk between the same endpoints. Repeating this always terminates at an honest path. **Consequence**: “does a u - v walk exist?” and “does a u - v path exist?” are the *same question* — but “shortest path” (easy, BFS) and “path of an exact length k ” (hard, in general) are genuinely different difficulty levels.

8.4 Connectedness is an equivalence relation

Define $u \simeq_G v$ iff a walk exists from u to v . This is reflexive (trivial length-0 walk), symmetric (reverse any walk), transitive (splice two walks together) — so it’s an equivalence relation, and its classes are exactly the **connected components** of G .

8.5 Two connectedness theorems

[Proof lives in your Main Proofs PDF] — item #5. If $|V(G)| \geq 3$ and both $G - u$ and $G - v$ are connected (for some two vertices u, v), then G itself is connected.

Application: any pair $x, y \neq u, v$ is already handled inside whichever of $G - u, G - v$ contains them both; for the pair $\{u, v\}$ itself, route through a third vertex w using a u - w path (inside $G - v$) spliced to a w - v path (inside $G - u$).

[Proof lives in your Main Proofs PDF] — item #6. (Ore-type.) If $d(u) + d(v) \geq n - 1$ for every nonadjacent pair u, v , then G is connected **and** $\text{diam}(G) \leq 2$. This bound $n - 1$ is the best possible (two disjoint edges on 4 vertices give every cross-pair sum $= 2 = n - 2$, one short — and that graph is disconnected).

Worked example (QB Day6 Q6). $n = 7$, every nonadjacent pair has $d(u) + d(v) \geq 6 = n - 1$ — directly matches the theorem above: G is connected and $\text{diam}(G) \leq 2$.

8.6 Other useful facts from this chapter (full proofs — short, worth knowing)

- **Edge-removal keeps connectivity iff that edge lies on a circuit.** ($G - e_i$ connected $\iff e_i$ lies on some circuit of G .) *Why:* if $e_i = (u, v)$ is on a circuit, the rest of the circuit is already an alternate u - v route once e_i is gone; conversely if $G - e_i$ is still connected, some u - v path avoiding e_i exists, and that path plus e_i forms a circuit.
- **Enough edges force connectivity:** a simple graph on n vertices with more than $\frac{(n-1)(n-2)}{2}$ edges **must** be connected (a disconnected graph's edges are maximised by one giant $(n-1)$ -clique plus one isolated vertex, which caps out at exactly $\frac{(n-1)(n-2)}{2}$ — so going even one edge over forces connectivity).
- **G and $\overline{G}$ can't both be disconnected.** If G splits into ≥ 2 components, then in $\overline{G}$ every cross-component pair becomes an edge, which is already enough to connect all of $\overline{G}$.

8.7 Common exam traps

- “No isolated vertices” $\neq$ “connected” (Chapter 8.1's counterexample).
- Distance is only defined *within* a connected graph (or within one component).
- The two boxed theorems above use *different* thresholds ($n-1$ here vs. n in Ore's Hamiltonicity theorem, Chapter 12) — don't mix them up.

Chapter 9 — Regular Graphs, Girth, Components & Instant Insanity

9.1 When does an r -regular graph on n vertices exist?

[Proof lives in your Main Proofs PDF] — item #7. Statement: for $0 \leq r \leq n - 1$, an r -regular graph of order n exists **iff at least one of r, n is even**. (One direction is one line: if both were odd, total degree nr would be odd, violating Handshaking.)

The construction you actually need (numerical skill — Harary graphs $H_{r,n}$):

1. Arrange the n vertices evenly around a circle.
2. If $r = 2k$ (even): join each vertex to its k nearest neighbours **clockwise** and k nearest **counter-clockwise**.
3. If $r = 2k + 1$ (odd, so n must be even): do step 2 for $2k$, **then** also join every vertex to its diametrically opposite vertex (adds exactly 1 to every degree).

Worked example. $H_{4,10}$: $r = 4 = 2(2)$, each vertex joined to its 2 nearest neighbours each side.

QB Day7 Q2: does a 3-regular graph exist on 5 vertices? $\sum d(v) = 15$, odd — **impossible**. On 6 vertices? $\sum d(v) = 18$, even, $\|G\| = 9$ — **exists** (e.g. the triangular prism: two triangles $\{1, 2, 3\}$, $\{4, 5, 6\}$ + rungs 1-4, 2-5, 3-6). **QB Day7 Q3:** $r = 3, n = 7$: $rn = 21$, odd — **no such graph**.

9.2 Embedding any graph inside a regular one (not skip-listed)

Theorem. For any G and any $r \geq \Delta(G)$, there's an r -regular graph H containing G as an induced subgraph. **Construction:** take a disjoint twin copy G' of G ; join every under-full vertex v_i (degree $< r$) to its twin v'_i . This raises the minimum degree by exactly 1 without ever exceeding r . Repeat $r - \delta(G)$ times.

9.3 Cycles, girth, components

- **Circuit:** closed walk. **Cycle:** circuit with $k \geq 3$ and no repeated vertex (except start=end).
- **Girth:** length of the graph's shortest cycle. (A tree has no cycle at all $\Rightarrow$ girth undefined/ ∞ .)
- **Component:** a maximal connected subgraph (= one equivalence class of $\simeq_G$, Chapter 8.4).

Theorem (not skip-listed — quick proof). G is disconnected $\iff V(G)$ splits into two nonempty disjoint sets V_1, V_2 with **no edge** between them. (Disconnected $\Rightarrow$ take V_1 = one component, V_2 = the rest; no edges cross components by definition. Conversely, if such a split exists, no walk can ever cross from V_1 to V_2 , so they're in different equivalence classes.)

9.4 Max edges with k components

[Proof lives in your Main Proofs PDF] — item #8.

$$\|G\| \leq \frac{(n-k)(n-k+1)}{2}$$

Achieved by: **one clique of size $n-k+1$ plus $(k-1)$ isolated vertices** — concentrate, don't spread out.

Worked example (QB Day7 Q4 style). $n = 10, k = 3$: max edges = $\frac{7 \times 8}{2} = 28$, achieved by K_8 plus 2 isolated vertices.

9.5 Exactly two odd-degree vertices $\Rightarrow$ a path joins them (not skip-listed)

If G has exactly two odd-degree vertices v_1, v_2 , a path joins them. (If they were in different components, each component would have exactly 1 odd-degree vertex — but Handshaking's corollary says every graph [including each component on its own] has an **even** count of odd-degree vertices. Contradiction.)

9.6 Instant Insanity (the Four-Cubes Puzzle)

Encoding. For each cube, build a graph on $\{R, B, G, Y\}$: join two colours with an edge whenever they sit on **opposite faces**. A valid stack needs **two edge-disjoint 2-regular subgraphs** (one per axis: Left/Right and Front/Back) of the 4-cube union-graph, each using **exactly one edge per cube**, so every colour appears exactly once on each of the tower's 4 long sides.

Worked example. With the four cube-graphs from your slides, one verified solution: $G_1 = \{R-G, B-R, Y-B, G-Y\}$ (Left/Right, one edge per cube) and $G_2 = \{Y-B, G-Y, B-R, R-G\}$ (Front/Back) — both the same 4-cycle $R-G-Y-B-R$, sourced from different cubes each time, so they're edge-disjoint per cube.

9.7 Bonus fact: $G - v$ is 3-regular $\Rightarrow |V(G)|$ is odd

If $G - v$ has m vertices and is 3-regular, $\sum d = 3m$ must be even (Handshaking) $\Rightarrow m$ even $\Rightarrow |V(G)| = m + 1$ **odd**. (Converse fails — a single triangle has odd order 3, but removing any vertex leaves a single edge, only 1-regular, not 3-regular.)

9.8 Common exam traps

- r -regular existence needs rn **even**, not r or n individually prime/small — always check parity first.
- Girth is **not** “grith” (a slide typo you’ll see referenced) — write “girth” on your exam.
- Max-edges-with- k -components is about the *best case*, not every disconnected graph — don’t apply it as an equality unless the graph is explicitly built to the extremal shape.

Chapter 10 — Euler Graphs, Unicursal Graphs & Fleury's Algorithm

10.1 Euler circuits and Euler's theorem

Definition. An **Euler circuit** is a closed walk using **every edge exactly once**. A graph possessing one is an **Euler graph**. (Connected — except isolated vertices are harmless, since the circuit just never visits them.)

[Proof lives in your Main Proofs PDF] — item #9.

G (connected) is an Euler graph $\iff$ every vertex has even degree

How to use it on a numerical question: compute every vertex's degree. All even + connected $\Rightarrow$ Euler circuit exists (and Fleury's algorithm, Section 10.3, finds it). Any odd degree anywhere $\Rightarrow$ no Euler circuit.

10.2 A closed trail always decomposes into cycles (not skip-listed)

Theorem. Every nontrivial closed trail is a union of edge-disjoint cycles. (*Idea: if the trail revisits a vertex, the loop between the two visits is itself a shorter closed trail/cycle; peel it off and induct on the remaining, shorter closed trail.*) **Worked example:** the “bowtie” (two triangles sharing vertex c), tour a, b, c, d, e, c, a splits exactly at the repeated vertex c into the two triangles $\{a, b, c\}$ and $\{c, d, e\}$.

10.3 Fleury's Algorithm — how to actually find the tour

[Proof lives in your Main Proofs PDF] — item #11 (that the algorithm always succeeds). The **procedure itself** is the numerical skill:

1. Pick any starting vertex (any vertex if finding a circuit; one of the two odd-degree vertices if finding a *trail*, Section 10.4).
2. Repeatedly walk along an edge incident to your current vertex and **delete it**.
3. **Never choose an edge whose removal would disconnect the remaining (non-isolated) graph** — unless it's your only option (“never burn a bridge too early”).
4. Stop when no edges remain.

Fully worked example. Graph on a, b, c, d, e, f with degrees $a=2, b=4, c=2, d=4, e=2, f=2$ (all even $\Rightarrow$ Euler graph). Running Fleury's from a :

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow b \rightarrow f \rightarrow d \rightarrow c \rightarrow a$$

— uses all 8 edges exactly once, no bridge ever forced early.

10.4 Unicursal graphs (Euler trails)

Definition. An open walk covering every edge exactly once is a **unicursal line / Euler trail**; a graph with one is **unicursal**.

[Proof lives in your Main Proofs PDF] — item #10.

G (connected) is unicursal $\iff$ exactly two vertices have odd degree

— and the Euler trail's two endpoints are **forced** to be exactly those two odd vertices.

Construction recipe: temporarily add an edge between the two odd vertices u, v (now every degree is even) $\Rightarrow$ find the Euler circuit (Fleury's) $\Rightarrow$ delete the added edge from the circuit $\Rightarrow$ what's left is your Euler trail from u to v .

Worked example (QB Day8 Q1). $G = (V, E)$, $V = \{a, b, c, d, e\}$, $E = \{be, bd, ed, ba, ac, ad, cd, bc\}$. Degrees: $a=3$ (edges ba, ac, ad), $b=4$, $c=3$, $d=4$, $e=2$. Two odd vertices $(a, c) \Rightarrow$ **not** an Euler graph, but **is unicursal** — trail from a to c , e.g. $a, d, e, b, d, c, a, b, c$ (verify: 8 edges, each used once).

10.5 Generalisation: $2k$ odd vertices $\Rightarrow k$ unicursal pieces (not skip-listed)

If a connected graph has exactly $2k$ odd-degree vertices, it splits into exactly k edge-disjoint unicursal pieces covering every edge. (*Pair up the $2k$ odd vertices arbitrarily, add k new edges to make everything even, find the Euler circuit, then delete the k added edges — each deletion increases the piece-count by 1, giving k pieces.*)

10.6 Arbitrarily traceable graphs

Definition. G is arbitrarily traceable from v if **every** edge-avoiding walk starting at v can always be extended to a full Euler tour — i.e. you can never trap yourself, no matter which unused edge you greedily pick.

Claim: arbitrarily traceable from $v \iff$ **every cycle of G passes through v .**

Worked example (the bowtie again). Every cycle ($\{a, b, c\}$ or $\{c, d, e\}$) passes through c — so c works: any greedy start from c finishes correctly (checked: all 4 first-move choices complete successfully). But a fails: $a \rightarrow b \rightarrow c \rightarrow a$ is a *legal* (not forced) greedy choice, yet it strands you at a with the whole $\{c, d, e\}$ triangle unreachable — because the triangle $\{c, d, e\}$ doesn't pass through a .

10.7 Join of two graphs — Eulerian condition (not skip-listed)

$G_1 + G_2$ (every vertex of G_1 joined to every vertex of G_2 , on top of each graph's own edges) is **always connected**. It is **Eulerian iff**:

$$\deg_{G_1}(v) \equiv |V(G_2)| \pmod{2} \text{ for every } v \in G_1, \text{ and symmetrically for } G_2$$

(Each G_1 -vertex gains exactly $|V(G_2)|$ new edges, so its new degree is $\deg_{G_1}(v) + |V(G_2)|$ — for this to be even, the two terms need matching parity.)

10.8 Common exam traps

- “Connected” is part of Euler's theorem's hypothesis — don't forget to check it, not just the degrees.
- A unicursal graph is **not** Eulerian (it has exactly 2 odd vertices, not zero) — don't call an Euler trail an “Euler circuit.”
- Fleury's bridge rule only forbids a choice when a *better* option exists — if several edges are all safe, any of them is fine (may give a different, equally valid, tour).

Chapter 11 — Hamiltonian Graphs

11.1 Definitions and quick one-directional facts

- **Hamiltonian cycle:** a cycle visiting **every** vertex exactly once. **Hamiltonian path:** a walk visiting every vertex exactly once (no return needed).
- Cycle $\Rightarrow$ path (delete one edge) but **not conversely** (P_4 has a trivial Hamiltonian path but no cycle at all).
- Hamiltonian $\Rightarrow$ connected, and $\Rightarrow$ no single vertex removal can disconnect it — but neither converse holds (Petersen graph, Section 11.6, is 3-connected yet **not** Hamiltonian).

11.2 Necessary condition — the component-counting test (mainly used to DISPROVE Hamiltonicity)

[Proof lives in your Main Proofs PDF] — item #12.

$$G \text{ Hamiltonian} \Rightarrow \kappa(G - S) \leq |S| \text{ for every nonempty proper } S \subseteq V(G)$$

($\kappa(G - S)$ = number of components after deleting S .)

How to use it (the numerical skill: disproving Hamiltonicity fast). Find **one** set S with $\kappa(G - S) > |S| \Rightarrow$ instantly **not Hamiltonian**, no search needed. **Worked example:** hub-like graph with $S = \{u, v, w\}$ and 4 leaves $\ell_1, \dots, \ell_4$ each attached to only one of u, v, w : removing S leaves 4 isolated leaves, $\kappa(G - S) = 4 > 3 = |S| \Rightarrow$ **not Hamiltonian**. Also instantly disqualifies any star $K_{1,m}$ ($m \geq 3$): $S = \{\text{hub}\}$ leaves m isolated leaves, $\kappa = m > 1 = |S|$.

11.3 Ore's Theorem

[Proof lives in your Main Proofs PDF] — item #13.

$$n \geq 3 \text{ and } d(u) + d(v) \geq n \text{ for every nonadjacent pair } u, v \Rightarrow G \text{ is Hamiltonian}$$

One direction only — the converse is **false** (C_5 : every nonadjacent pair sums to $2 + 2 = 4 < 5 = n$, yet C_5 is trivially Hamiltonian).

Worked example. K_4 minus edge ab : only nonadjacent pair is (a, b) , $d(a) + d(b) = 2 + 2 = 4 \geq n = 4$ OK — Ore's theorem guarantees Hamiltonicity, and indeed a, c, b, d, a is a Hamiltonian cycle.

Worked capstone (join, QB Day10-style). G 3-regular, order 12; H 4-regular, order 11; $n = |G + H| = 23$. Only possible nonadjacent pairs are *within* G or *within* H (every G - H pair is adjacent by definition of join). Within- G pair: d -sum = $(3 + 11) + (3 + 11) = 28 \geq 23$. Within- H pair: $(4 + 12) + (4 + 12) = 32 \geq 23$. Every nonadjacent pair clears Ore's threshold $\Rightarrow$ **Hamiltonian**.

11.4 Closure Theorem

[Proof lives in your Main Proofs PDF] — item #14.

$$G \text{ is Hamiltonian} \iff C(G) \text{ is Hamiltonian}$$

Building the closure (numerical construction skill): repeatedly pick any nonadjacent pair with $d(u) + d(v) \geq n$ at that moment and add the edge; keep going until no such pair remains, or you reach K_n . **Why it's useful:** if $C(G) = K_n$, you're instantly done (K_n is trivially Hamiltonian for $n \geq 3$) $\Rightarrow G$ was Hamiltonian all along — often much faster than checking Ore's condition on G directly.

11.5 Degree-Counting Theorem (a second sufficient condition)

[Proof lives in your Main Proofs PDF] — item #15.

if, for every $1 \leq j < \frac{n}{2}$, $\#\{v : \deg(v) \leq j\} < j \implies G$ is Hamiltonian

Beginner translation, one j at a time (this is the trick to actually apply it fast):

- $j = 1$: **zero** vertices may have degree ≤ 1 (no isolated or pendant vertices at all).
- $j = 2$: **at most one** vertex may have degree ≤ 2 .
- $j = 3$: **at most two** vertices may have degree ≤ 3 . — and so on.

Low-degree vertices are allowed, just not in large numbers. Converse is false — same culprit graph, C_5 : $n = 5$ needs $j = 1, 2$ checked; at $j = 2$, **all 5** vertices have degree ≤ 2 , but the rule needs < 2 — fails, yet C_5 is Hamiltonian anyway.

11.6 The Petersen graph — a graph where “nice” invariants can still fail Hamiltonicity

3-regular, 10 vertices, 15 edges, girth 5 — a textbook example that is **highly connected yet not Hamiltonian** (no Hamiltonian cycle exists, verified exhaustively), though it **does** have a Hamiltonian **path**. Good reminder: strong local structure (regularity, connectivity) still doesn’t guarantee a Hamiltonian cycle.

11.7 Counting Hamiltonian cycles that decompose K_n (odd n) — not skip-listed, full construction

Upper bound (quick counting): K_n has $\binom{n}{2} = \frac{n(n-1)}{2}$ edges; each Hamiltonian cycle uses n of them, so at most $\frac{n-1}{2}$ edge-disjoint Hamiltonian cycles can possibly fit.

Matching construction (achieves the bound exactly, for odd n): fix one vertex v_0 ; place the other $m = n - 1$ (even) vertices $1, \dots, m$ around a circle. Zigzag: $v_0, 1, m, 2, m-1, 3, m-2, \dots, v_0$ (alternate smallest/largest remaining label) — this uses every “chord length” exactly once. Rotate the labels by $\frac{360^\circ}{m}, \frac{2 \cdot 360^\circ}{m}, \dots$ (keeping v_0 fixed) to get $\frac{n-3}{2}$ more, edge-disjoint, Hamiltonian cycles. Total: $\frac{n-3}{2} + 1 = \frac{n-1}{2}$ — exactly the upper bound, so this decomposes **all** of K_n ’s edges.

K_n (odd $n \geq 3$) decomposes into exactly $\frac{n-1}{2}$ edge-disjoint Hamiltonian cycles

Worked example (K_5). Predicted $\frac{5-1}{2} = 2$ cycles. Pentagon $0, 1, 2, 3, 4, 0$ (5 “short” edges) + pentagram $0, 2, 4, 1, 3, 0$ (5 “long” edges) — disjoint, together exactly all $\binom{5}{2} = 10$ edges. OK

Worked example (K_9 , QB Day10 Q3). $\frac{9-1}{2} = 4$ cycles, each using 9 edges: $4 \times 9 = 36 = \binom{9}{2}$. OK

Why n must be odd: if n is even, every vertex of K_n has degree $n - 1$, which is **odd** — but a Hamiltonian cycle only ever contributes an *even* amount (exactly 2) to each vertex’s degree, so whole Hamiltonian cycles alone can never cleanly cover an odd leftover degree.

11.8 Common exam traps

- Ore’s threshold is $\geq n$ (Hamiltonicity) — don’t confuse with $\geq n - 1$ (mere connectivity + diam ≤ 2 , Chapter 8.5).

- The component-counting test (11.2) only **disproves** Hamiltonicity when it fails — passing it for every S you happen to check is **not** a proof of Hamiltonicity.
- Ore's theorem, the closure theorem, and the degree-counting theorem are all **sufficient, not necessary** — C_5 is the standard counterexample for all three, keep it in your back pocket.

Chapter 12 — Master Formula Sheet (last-minute lookup)

#	Topic	Formula / Rule
1	Handshaking (recap)	$\sum_v d(v) = 2\ G\ $; odd-degree vertex count is even
2	Max simple-graph edges	$\binom{n}{2} = \frac{n(n-1)}{2}$
3	Union counting	$ V_1 \cup V_2 = n_1 + n_2 - V_1 \cap V_2 $; $\ G_1 \cup G_2\ = m_1 + m_2 - \ G_1 \cap G_2\ $
4	Ring-sum counting	$\ G_1 \oplus G_2\ = m_1 + m_2 - 2\ G_1 \cap G_2\ $
5	Cartesian product counting	$ V(G_1 \boxtimes G_2) = n_1 n_2$; $\ G_1 \boxtimes G_2\ = n_1 m_2 + n_2 m_1$
6	Isomorphism necessary checks	same $ V $, $ E $, degree sequence (never sufficient)
7	Decomposition count	genuine 2-piece decompositions of m edges $= 2^{m-1} - 1$
8	Degree sandwich	$0 \leq \delta(G) \leq \deg(v) \leq \Delta(G) \leq n - 1$
9	Average degree	$\bar{d}(G) = \frac{2\ G\ }{n} \geq 2k \Rightarrow \exists F$ induced, $\delta(F) \geq k + 1$
10	Graphical quick checks	sum even; max entry $\leq n - 1$ (both necessary, neither sufficient)
11	Havel-Hakimi	remove d_1 , subtract 1 from next d_1 entries, re-sort, repeat
12	Regular-graph existence	r -regular on n exists $\iff r$ or n even
13	Max edges, k components	$\ G\ \leq \frac{(n-k)(n-k+1)}{2}$
14	Odd-pair connectivity	exactly 2 odd-degree vertices $\Rightarrow$ a path joins them
15	Euler's theorem	connected + all-even degree $\iff$ Euler circuit exists
16	Unicursal theorem	connected + exactly 2 odd-degree vertices $\iff$ Euler trail exists
17	Join Eulerian condition	Eulerian $\iff \deg_{G_1}(v) \equiv V(G_2) \pmod{2}$ both ways
18	$2k$ -odd generalisation	$2k$ odd-degree vertices $\Rightarrow$ splits into k unicursal pieces
19	Hamiltonian necessary test	Hamiltonian $\Rightarrow \kappa(G - S) \leq S \forall S$ (use contrapositive to disprove)
20	Ore's theorem	$d(u) + d(v) \geq n \forall$ nonadjacent pairs $\Rightarrow$ Hamiltonian
21	Closure theorem	G Hamiltonian $\iff C(G)$ Hamiltonian
22	Degree-counting theorem	$\#\{v : \deg(v) \leq j\} < j \forall 1 \leq j < n/2 \Rightarrow$ Hamiltonian

#	Topic	Formula / Rule
23	K_n Hamiltonian decomposition	odd $n \geq 3$: decomposes into $\frac{n-1}{2}$ edge-disjoint Hamiltonian cycles
24	SSP pairwise weight	$d(s_i, s_j) = s_j - \text{overlap}(s_i, s_j)$; shortest superstring = min-weight Hamiltonian path

The five sufficient-not-necessary theorems to remember with C_5 as the standard counterexample: two-equal-degree theorem's cousin conditions, Ore's Theorem, the Closure Theorem (trivially, since it wraps Ore), the Degree-Counting Theorem, and the component-counting necessary test (which is necessary, not sufficient, the *other* way round — don't mix up which direction each one fails in).

The seventeen proofs living in your separate Main Proofs PDF (state + apply here; derive there): Handshaking Lemma · odd-degree count is even · two vertices share a degree · Havel-Hakimi · $G-u, G-v$ connected $\Rightarrow G$ connected · degree-sum $\Rightarrow$ connected + $\text{diam} \leq 2$ · r -regular existence · max edges with k components · Euler's theorem · unicursal theorem · Fleury's algorithm correctness · Hamiltonian $\Rightarrow \kappa(G-S) \leq |S|$ · Ore's theorem · closure theorem · degree-counting theorem · infinite graph with finite vertices (pigeonhole) · vertex-transitive $\Rightarrow$ regular.

End of summary. Good luck tomorrow — you've now covered every topic in pages 1-194 (Day 1 through Day 10) with a formula and a worked numerical example attached to it.